Repo Architecture Mapping - Reusable Prompt

Purpose: Drop this into a Claude Code session pointed at any repo. It dispatches a team of subagents to explore the codebase in parallel, then generates an interactive HTML architecture map with clickable links to the actual source files.

Where it came from: This is the generic, repo-agnostic version of the prompt used in Demo 1 of the ITSS workshop (where Claude Code mapped Codex CLI and anomalyco/opencode side-by-side). Strip the Codex/opencode specifics, keep the structure that worked, and you get a tool that lands on any codebase your team owns.

Why parallel subagents: Single-agent exploration takes 5-10 minutes on a substantial repo and produces a flat narrative. Parallel subagents each get a focused scope (entry points, business logic, data layer, integrations, infrastructure) and return concentrated reports. The orchestrator synthesizes them into a layered map. Better artifacts, similar wall-clock time.

Works with /goal: the prompt is structured around a single goal statement; set it as the session goal, then paste the body, and Claude Code's goal-tracking will follow progress as the orchestrator dispatches and the subagents return.

How to use

Step 1 - Set the goal (optional but recommended)

Inside a Claude Code session in the target repo:

```
/goal Map this repo's architecture into an interactive HTML using parallel subagents, with
file:line citations for every claim and clickable links to the actual source files.
```

Step 2 - Paste the prompt

Copy the entire block below into the Claude Code session. The agent will read GOAL + APPROACH and start dispatching.

The prompt

```
Produce an interactive HTML architecture map of this repository.
Parallel subagent exploration, clickable file:// links to every cited source location, and a
validation pass at the end.
APPROACH (orchestrator -> subagents -> orchestrator):
1. ORCHESTRATOR FIRST PASS.
   Read the top-level directory listing, the README, and the primary
   package manifest you find (package.json, Cargo.toml, pyproject.toml,
   go.mod, pom.xml, build.gradle, etc.). Identify primary language(s),
   framework(s), license, and the high-level shape of the codebase.
2. DECOMPOSE INTO 3-5 EXPLORATION SCOPES.
```

```
   Pick what fits THIS repo. Default scopes (adjust based on what you
   actually see):
     - Entry points + bootstrap (CLI mains, server starts, top-level scripts)
     - Core business logic / domain modules
     - Data layer / persistence / state management
     - Integration surfaces (API clients, external services, MCP / tool
       integrations, message queues)
     - Infrastructure (build, test, deployment, CI, IaC)
   If the repo is small (<5k LOC) or single-purpose (a CLI tool, a
   library), use 2-3 scopes instead of 5. If it's a monorepo, use one
   scope per package or one per service.
3. DISPATCH ONE SUBAGENT PER SCOPE via the Task tool.
   Each subagent gets:
     - Scope name + one-sentence purpose
     - 1-3 starting directories or files
     - Output contract: structured markdown with a table per module -
       "Module | Purpose (one sentence) | Key files (file:line) |
       Notable patterns | Open questions"
     - Quality bar: every claim has a file:line citation. No claims
       without citations. Don't invent paths. If something is unclear,
       flag it as an Open Question rather than guessing.
4. SYNTHESIZE WHEN ALL RETURN.
   - Resolve overlaps: if two subagents touched the same file, prefer
     the more specific one.
   - Note any disagreements as Open Questions.
   - Build a single coherent architecture model.
5. GENERATE INTERACTIVE HTML AT ./docs/architecture-map.html.
   Required structure:
   - Header block: repo name (from git remote or README), primary
     language(s), license (read from LICENSE if present), generation
     timestamp.
   - One section per scope, color-coded with distinct backgrounds.
   - Per module: name, one-line purpose, key files as clickable
     file:// links using ABSOLUTE paths to this repo (use the
     current working directory's absolute path as the prefix).
   - Cross-cutting patterns section (if multiple subagents observed
     the same pattern across scopes - e.g., dependency injection style,
     async pattern, error-handling convention).
   - Footer: list of top-level directories/files NOT covered (so the
     reader knows what's uncharted), plus all Open Questions surfaced
     by subagents.
   - Simple HTML + CSS, no external dependencies, no JS frameworks.
     Must open standalone in a browser.
6. VALIDATE EVERY file:// LINK.
   Walk the generated HTML, extract every href="file:///...",
   confirm the target exists on disk. Fix or remove any broken link.
   Report the total link count and resolve rate (e.g., "73/73 links
   resolve, 100%").
CONSTRAINTS:
- Read-only on the repo. Don't modify any source files.
- If the repo is >100k LOC, scope each subagent to a specific
  subdirectory rather than a whole concern. Otherwise the subagents
  return too much.
- If the repo has no recognizable structure (e.g., research code,
  abandoned experiments), say so and stop. Don't invent architecture.
- Use standard file:// links: absolute paths, no trailing
  punctuation, no spaces (URL-encode if needed). Note in the
  generated HTML footer that Chrome (and Edge) block file:// -> file://
  nav by default; launch with --allow-file-access-from-files and an
  isolated --user-data-dir profile to enable clickthrough.
```

What you get back

A single HTML file you can open in a browser. Chrome (and Edge) block file:// -> file:// navigation by default - launch with --allow-file-access-from-files --user-data-dir=/tmp/chrome-arch to enable clickthrough into source files (the /tmp/ profile isolates the security-relaxed session from your normal browsing). Sections per scope, modules with one-line purposes, clickable links to actual source files, and an honest "what's uncharted" footer.

Typical numbers (from our workshop simulation on real repos):

- Codex CLI (mostly Rust, dozens of crates): 554-line HTML, 73 file:// links, 100% resolve rate
- anomalyco/opencode (mostly TypeScript monorepo): 775-line HTML, 100 file:// links, 99% resolve rate (one path hallucinated)

The bottleneck is wall-clock time: ~8-10 minutes for an average-sized repo, more for monorepos.

How to customize

The prompt has a few knobs worth tuning per use case. Search/replace these:

Knob	Default	When to change
Output filename	./docs/architecture-map.html	If you want multiple maps in the same repo (per-module, per-team) - change to ./docs/architecture-<scope>.html, or drop into the repo root if the team doesn't keep a /docs directory
Scope count	3-5 subagents	Small CLI / library: 2-3. Big monorepo: one per package.
Default scopes	Entry / business logic / data / integrations / infrastructure	Replace with domain-specific scopes if you're in a specific stack (e.g., frontend / backend / shared types / build for a typical SPA repo)
LOC threshold for sub-directory mode	100k	Lower if your runtime is constrained. Raise if you're running on a beefy machine and want bigger scope per subagent.
Browser launch wrapper	google-chrome --allow-file-access-from-files --user-data-dir=/tmp/chrome-workshop	Put a chrome-workshop launcher on PATH (see "WSL2 / Linux setup" section below) so your team launches the map with one word. The isolated /tmp/chrome-workshop profile keeps the security-relaxed Chrome separate from normal browsing.

WSL2 / Linux setup -- the chrome-workshop wrapper

Chrome and Edge block file:// -> file:// navigation by default, which means the architecture map's clickable links won't work without launch flags. The workshop pattern is to wrap the launch in a one-word command. Create ~/.local/bin/chrome-workshop once and make it executable:

```
#!/usr/bin/env bash
exec google-chrome --allow-file-access-from-files --user-data-dir=/tmp/chrome-workshop "$@"
```

After that, chrome-workshop ./docs/architecture-map.html opens the map with file:// clickthrough working. The /tmp/chrome-workshop profile dir isolates this security-relaxed

Chrome from your normal browsing -- nothing leaks into your normal profile and the isolated profile can be deleted any time with rm -rf /tmp/chrome-workshop.

Why a launcher and not an alias: a shell script on PATH works in both interactive and noninteractive checks, passes arguments cleanly (chrome-workshop file1.html file2.html opens two), and can be extended later (e.g., add --incognito for one-shot opens).

On macOS: swap google-chrome for open -a "Google Chrome" --args or use Chromium directly. On Windows-native (no WSL2): use PowerShell Start-Process chrome -ArgumentList ... -- but most of you are on WSL2, so the PATH launcher is the canonical recipe.

Failure modes + recovery

Failure	Symptom	Recovery
Live generation too slow	Past 15 min, still no HTML	Cancel (Ctrl-C). Re-run with smaller scope count (2 subagents instead of 5). Or split the repo and run twice.
Hallucinated file paths	Validation step reports broken links	The prompt's validation step catches most of these. If many slip through, open the HTML and delete the offending entries by hand - usually a small fraction.
Repo too large for one pass	Subagent context windows fill up, return incomplete	Use the "subdirectory mode": each subagent gets a specific subtree instead of a cross-cutting concern.
Repo too small or unstructured	Prompt returns "no recognizable structure"	That's correct behavior. The prompt is honest when there's nothing to map.
file:// links don't navigate in browser	Chrome/Edge blocks file:// -> file:// by default	Launch Chrome with --allow-file-access-from-files --user-data-dir=/tmp/chrome-arch (the isolated profile keeps the security-relaxation contained).

How this maps to the workshop

This prompt is the take-home version of what you saw in Demo 1, with one important orchestration difference. In the workshop, Mihai ran two parallel Claude Code instances -- a Codex-specific variant in Pane A and an opencode-specific variant in Pane B -- side-by-side, two isolated CLI sessions on different repos. This take-home prompt uses parallel subagents inside a single Claude Code instance (dispatched via the Task tool, step 3 of the prompt) on one repo. Both are valid parallel-exploration patterns. The two-instance pattern from the demo lets you compare two codebases head-to-head; this take-home subagent pattern lets you decompose one codebase across 3-5 exploration scopes concurrently. Pick the pattern that fits your task: cross-repo comparison -> two instances; single-repo decomposition -> subagents.

Apply it on Monday to:

- A legacy repo nobody on the team fully remembers
- A microservice you're about to inherit
- An open-source library you're vendor-evaluating
- A monorepo where you want a high-level map for the docs site

The output HTML is the artifact your team takes back. The methodology - parallel subagents, file:line discipline, validation pass - is the lesson that transfers across any future tooling.

ITSS Workshop - 28 May 2026. Take-home prompt. Free to copy, modify, redistribute internally.